VISVESVARAYA TECHNOLOGICAL UNIVERSITY

"JNANA SANGAMA" Belagavi – 590018, Karnataka

A

MACHINE LEARNING PROJECT REPORT

ON

"**AcadeScore: A Random Forest Machine Learning Framework for Tracking Student Performance Based on Lifestyle Factors**"

Submitted in partial fulfillment for the award of the degree in

BACHELOR OF ENGINEERING

IN

COMPUTER SCIENCE AND ENGINEERING

Submitted by

JAYALAKSHMI TH	1BH23CS030
LAVANYA G	1BH23CS040
MUSHARRAF ALI	1BH23CS053

Under the Guidance of

Mrs. Shylaja D N

Asst. Prof., Dept. of CSE

BTI BANGALORE TECHNOLOGICAL INSTITUTE

Approved by AICTE, New Delhi | Affiliated to VTU, Belagavi | Accredited by NAAC | ISO Certified
Kodathi Village, Varthoor Hobli, Bengaluru East Taluk, Bengaluru Urban District,
Bengaluru , Karnataka – 560035

www.btibangalore.org | principal@btibangalore.org | 7090404050

Department of Computer Science and Engineering

BANGALORE TECHNOLOGICAL INSTITUTE

(NAAC Accredited, An ISO 9001:2005 Certified Institute)

Kodathi Village, Varthur Hobli, Bangalore East Tq, Bangalore Urban District

Bangalore – 560035, Karnataka

2025 – 2026

BTI BANGALORE TECHNOLOGICAL INSTITUTE
Approved by AICTE, New Delhi | Affiliated to VTU, Belagavi | Accredited by NAAC | ISO Certified
Kodathi Village, Varthoor Hobli, Bengaluru East Taluk, Bengaluru Urban District,
Bengaluru , Karnataka – 560035
www.btibangalore.org | principal@btibangalore.org | 7090404050

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

CERTIFICATE

This is to certify that the Machine Learning Project report on **"AcadeScore: A Random Forest Machine Learning Framework for Tracking Student Performance Based on Lifestyle Factors"** caried out by **Jayalakshmi T H [1BH23CS030], Lavanya G [1BH23CS040], Musharraf Ali [1BH23CS053]** the Bonafide student of **Bangalore Technological Institute**, Bengaluru in partial fulfilment for the award of **Bachelor of Engineering** in **Computer Science and Engineering** of **Visvesvaraya Technological University, Belagavi** during the academic year **2025 -2026.** Thus, it is certified that all corrections/suggestions indicated for Internal Assessment have been incorporated in the report. The Mini Project report has been approved, as it satisfies the academic requirements in the respect of the Mini Project report prescribed for the said degree.

Details of the Batch	Context & clarity (12)	Technical Depth & Architecture (13)	Project Implementation & Execution (12)	Presentation & Viva (13)	Report (25)	Total (75)	Final (25)
Jayalakshmi TH (1BH23CS030)							
Lavanya G (1BH23CS040)							
Musharraf Ali (1BH23CS053)							

Mrs. Shylaja D N
Assistant Professor
Department of CSE

Dr. D. Jerline Sheebha Anni
Professor & Head
Department of CSE

ABSTRACT

AcadeScore is a data-driven web application developed to monitor, analyze, and enhance student academic performance by considering multiple behavioral and lifestyle factors such as study hours, attendance, sleep patterns, social media usage, mental health, and part-time employment. The system leverages a machine learning approach, specifically the Random Forest regression model, to predict a comprehensive performance score based on user-input data. Random Forest is chosen for its ability to handle non-linear relationships, reduce overfitting, and provide feature importance insights. The model is trained on structured datasets and evaluated using standard performance metrics such as Mean Absolute Error (MAE), Mean Squared Error (MSE), and R^2 score to ensure prediction accuracy and reliability. The application presents results through an interactive dashboard that visualizes performance trends, consistency scores, and influencing factors using charts and graphs. Additionally, it includes an improvement tracking system and generates personalized suggestions and focus areas based on recent data patterns. By combining machine learning, statistical analysis, and intuitive visualization, AcadeScore provides students with actionable insights to improve productivity, maintain balance, and achieve better academic outcomes.

INDEX

Abstract	i
Content	ii-iii
List of Figures	iv
List of Tables	v

TABLE OF CONTENTS

SI.No	Chapter	Page No.
01	Introduction	1
	1.1 Overview	1
	1.2 Background of Student Performance Prediction	2
	1.3 Technology Used	2-3
	1.4 Problem Statement	3
	1.5 Objectives of the Project	3-4
	1.6 Motivation	4
02	Literature Survey	5
03	System Analysis	6
	3.1 Existing System	6
	3.2 Advantages of Existing System	6
	3.3 Disadvantages of Existing System	6-7
	3.4 Proposed System	7
	3.5 Feasibility Study	7-8
	3.6 Comparison: Existing and Proposed System	8
04	Objectives and Methodology	9
	4.1 Objectives	9
	4.2 Methodology	9-11
05	Requirement Analysis	12
	5.1 Introduction	12
	5.2 Hardware Requirements	12
	5.3 Software Requirements	13
	5.4 Functional Requirements	14
	5.5 Non-Functional Requirements	15

06	Implementation	16
	6.1 Introduction	16
	6.2 Machine Learning Algorithm Used	16-17
	6.3 System Architecture	17-18
	6.4 Implementation Procedure	18-19
07	Results and Discussion	20
	7.1 Introduction	20
	7.2 System Execution	20
	7.3 Output Results	20-21
	7.4 Performance Analysis	21
	7.5 Discussion of Results	21
08	Conclusion	22
	8.1 Conclusion	22
	8.2 Future Enhancements	22-23
	REFERENCES	24
	APPENDIX A: SCREENSHOTS	25-27
	APPENDIX B: SOURCE CODE	28-34
	APPENDIX C: PHOTO GALLERY	35-36

LIST OF FIGURES

Fig. No	Title of the Figure	Page No.
6.1	AcadeScore System Architecture	18

LIST OF TABLES

Table No.	Title of the Table	Page No.
2.1	Literature Survey Comparison	5
3.1	Comparison (Existing System, Proposed System)	8

Chapter 1

INTRODUCTION

1.1 Overview

In the present digital era, education has become one of the most important pillars for personal and professional development. Academic success is not solely determined by intelligence but is influenced by a variety of factors such as study habits, attendance, lifestyle choices, mental well-being, and time management. However, traditional education systems primarily evaluate student performance based on examination results, which do not provide insights into the underlying factors affecting performance.

This limitation creates a gap between performance evaluation and performance improvement. Students often receive feedback only after examinations, which is too late to make meaningful improvements. Therefore, there is a growing need for systems that can analyze student behaviour and predict performance in advance. The system uses the Random Forest Regression algorithm to analyze input data such as study hours, attendance, sleep patterns, and mental health ratings.

With the advancement of machine learning and data analytics, it is now possible to analyze large volumes of data and identify patterns that influence outcomes. Machine learning models can process multiple variables simultaneously and generate predictions with high accuracy. These capabilities can be applied in the educational domain to predict student performance and provide actionable insights.

The proposed project, titled **"AcadeScore"**, aims to develop a machine learning-based application that predicts student performance based on academic and lifestyle factors. The system uses the Random Forest Regression algorithm to analyze input data such as study hours, attendance, sleep patterns, and mental health ratings.

The application provides a user-friendly web interface where students can input their data and receive predicted scores. This enables students to understand how their habits affect their performance and take corrective actions before examinations.

Thus, the project integrates machine learning with web technologies to create an intelligent system that enhances student learning and academic success.

1.2 Background of Student Performance Prediction

In The field of education has undergone significant transformation with the integration of technology. Traditional teaching methods are now being supplemented with data-driven approaches that aim to improve student outcomes. Educational data mining and learning analytics have emerged as important research areas that focus on analyzing student data to extract meaningful insights.

Machine learning plays a crucial role in this transformation. It enables systems to learn from historical data and make predictions about future outcomes. In the context of education, machine learning can be used to predict student performance, identify at-risk students, and provide personalized learning recommendations.

Several machine learning algorithms have been applied in this domain, including Linear Regression, Decision Trees, Support Vector Machines, and Random Forest. Among these, Random Forest has proven to be highly effective due to its ability to handle complex datasets and reduce overfitting.

Random Forest is an ensemble learning technique that combines multiple decision trees to produce more accurate predictions. It works by creating multiple trees using random subsets of data and features, and then averaging their outputs. This improves both accuracy and stability.

In this project, student data is used to train the Random Forest model. The dataset includes both academic and lifestyle features, providing a comprehensive view of student behaviour. By analysing these features, the model can predict student performance more accurately compared to traditional methods.

1.3 Technology Used

The development of the AcadeScore system involves the use of several technologies, each playing a specific role in the system architecture.

Machine Learning

Machine learning is the core component of this project. It is used to build predictive models that analyze student data and generate performance predictions.

Random Forest Regression

Random Forest Regression is the primary algorithm used in this project. It is chosen because of its high accuracy, ability to handle multiple features, and robustness against overfitting.

Python

Python is used as the main programming language due to its simplicity and extensive support for machine learning libraries.

Flask Framework

Flask is a lightweight web framework used to develop the backend of the application. It handles user requests, processes input data, and communicates with the machine learning model.

HTML, CSS, and JavaScript

These technologies are used to design the frontend interface. They provide a user-friendly environment where users can enter data and view results.

1.4 Problem Statement

In the current educational system, student performance is evaluated primarily based on examination results. While this approach provides a measure of academic achievement, it does not offer insights into the factors that influence performance.

Students often struggle to understand how their daily habits, such as study time, sleep patterns, and attendance, affect their academic outcomes. As a result, they are unable to make informed decisions to improve their performance.

Additionally, there is no system that provides early prediction of student performance. Feedback is usually given after examinations, which limits the opportunity for improvement.

Therefore, there is a need for an intelligent system that can analyze student data and predict performance in advance. Such a system would enable students to identify weaknesses, adopt better habits, and improve their academic results.

1.5 Objectives of the Project

The primary objectives of the AcadeScore project are:

- To develop a machine learning based system that predicts student scores using lifestyle factors.

- To collect and preprocess student-related data for training a predictive model.
- To implement a Random Forest Regression model for accurate score prediction.
- To evaluate the model using performance metrics such as MAE, RMSE, and R^2 score.
- To analyze feature importance in order to determine which lifestyle factors influence academic performance the most.
- To design a user-friendly web application that allows students to input their details and obtain predictions instantly.
- To demonstrate the practical application of machine learning in educational analytics

1.6 Motivation

The motivation behind developing AcadeScore arises from the growing importance of intelligent educational tools that support student success. Many students are unaware of how their lifestyle habits directly affect their academic outcomes. Without proper guidance, they often realize their weaknesses only after receiving poor results.

The motivation behind this project stems from the increasing importance of data-driven decision-making in education. With the growing complexity of academic environments, students require better tools to understand and improve their performance.

Machine learning provides an opportunity to analyze complex datasets and extract meaningful insights. By applying machine learning techniques to student data, it is possible to create systems that provide personalized feedback and support.

This project aims to bridge the gap between technology and education by developing a system that not only predicts performance but also helps students take proactive steps towards improvement.

Chapter 2

LITERATURE SURVEY

Table 2.1: Literature Survey Comparison

Sl. No	Title	Author / Year	Technique Used	Key Contribution
1	Predicting University Student Graduation Using Academic Performance and Machine Learning: A Systematic Literature Review	Pelima et al. (2024)	SVM, Random Forest, Ensemble ML	Reviews ML methods and identifies research gaps
2	Early Predicting of Students Performance in Higher Education	Alhazmi & Sheneamer (2023)	T-SNE, Classification Models	Early prediction using academic and admission data
3	Utilizing Machine Learning Models to Predict Student Performance From LMS Activity Logs	Khan et al. (2023)	Data Mining, Neural Networks	Uses LMS activity data for performance prediction
4	Multiclass Prediction Model for Student Grade Prediction Using Machine Learning	Bujang et al. (2021)	SVM, RF, NB, kNN + SMOTE	Improves prediction accuracy for imbalanced data
5	Predicting Student Performance and Its Influential Factors Using Hybrid Regression and Multi-Label Classification	Alshanqiti & Namoun (2020)	Hybrid ML + SOM	Identifies factors affecting student performance

Chapter 3

SYSTEM ANALYSIS

3.1 Existing System

In the current educational framework, student performance is primarily evaluated based on examination results. The system relies on periodic tests, assignments, and final exams to measure student progress.

Although this method provides a standard way of assessing students, it has several limitations. It focuses only on outcomes rather than the factors influencing those outcomes. The existing system does not consider important aspects such as:

- Study habits
- Time management
- Sleep patterns
- Mental health
- Lifestyle behaviour

As a result, students receive feedback only after examinations, which limits their ability to improve in advance.

3.2 Advantages of Existing System:

Despite its limitations, the existing system has certain advantages:

- Simplicity: Easy to implement and understand
- Standardization: Provides a uniform method of evaluation
- Wide Adoption: Used globally in educational institutions
- Structured Assessment: Based on predefined syllabus and exams

These advantages make the system reliable, but they do not address the need for predictive insights.

3.3 Disadvantages of Existing System

The traditional evaluation system has several drawbacks:

Lack of Prediction

The system does not provide any prediction about future performance. Students only know their results after exams.

No Early Intervention

There is no mechanism to identify weak students early and help them improve.

Limited Analysis

The system does not analyze factors affecting performance such as lifestyle and behavior.

No Personalization

Every student is evaluated in the same way, without considering individual differences.

Delayed Feedback

Feedback is provided only after exams, which is often too late for improvement.

3.4 Proposed System (AcadeScore Model):

To overcome the limitations of the existing system, a machine learning-based solution is proposed.

The proposed system, AcadeScore, predicts student performance based on multiple input features such as:

- Study hours
- Attendance percentage
- Sleep duration
- Social media usage
- Mental health rating

The system uses the Random Forest Regression algorithm to analyze these features and predict the student's exam score.

The system is implemented as a web application using Flask, allowing users to input their data and receive predictions in real time.

3.5 Feasibility Study:

3.5.1 Technical Feasibility

The system is technically feasible because:

- Uses Python and ML libraries
- Requires no complex hardware
- Easy to implement and maintain

3.5.2 Operational Feasibility

The system is user-friendly and easy to operate:

- Simple interface
- No technical knowledge required
- Accessible to students

3.5.3 Economic Feasibility

The system is cost-effective:

- Uses open-source tools
- No additional hardware required
- Low development cost

3.6 Comparison: Existing vs Proposed System

Table. 3.1: Comparison: Existing vs Proposed System

Feature	Existing System	Proposed System
Prediction	No	Yes
Analysis	Limited	Detailed
Feedback	Late	Early
Personalization	No	Yes
Technology	Manual	ML-based

Chapter 4

OBJECTIVES AND METHODOLOGY

4.1 Objectives

The primary objective of this project is to design and develop a machine learning-based system that can accurately predict student performance based on various academic and lifestyle factors.

The detailed objectives are as follows:

- **Performance Prediction**
  To develop a predictive model that can estimate student exam scores based on input data such as study hours, attendance, and lifestyle habits.
- **Multi-Factor Analysis**
  To analyse multiple features including academic and non-academic factors and understand their impact on performance.
- **Early Feedback System**
  To provide early prediction results so that students can identify weaknesses and improve before examinations.
- **User-Friendly System**
  To design an intuitive web-based interface that allows users to input data and obtain results easily.
- **Accuracy and Efficiency**
  To ensure the model produces accurate predictions with minimal computational complexity.
- **Real-World Application**
  To demonstrate the practical application of machine learning in the field of education.

4.2 Methodology

The development of this project follows a systematic approach that includes data processing, model training, evaluation, and deployment.

4.2.1 Data Collection:

Data collection is the first and most important step in building a machine learning model.

In this project, the dataset contains information related to student behavior and performance. The dataset includes both academic and lifestyle features such as:

- Study hours per day
- Attendance percentage
- Sleep duration
- Social media usage
- Mental health rating

The dataset used is stored in a CSV file named:

student_habits_performance.csv

This dataset provides sufficient information to train the model and identify patterns affecting student performance.

4.2.2 Data Preprocessing

Raw data is often incomplete, inconsistent, or noisy. Therefore, preprocessing is required before training the model.

The preprocessing steps include:

i. **Data Cleaning**
   - Removing missing or invalid values
   - Ensuring consistency in data

ii. **Data Formatting**
   - Converting categorical data into numerical format
   - Normalizing values if required

iii. **Feature Selection**
   - Selecting relevant features that influence performance
   - Removing unnecessary or redundant data

4.2.3 Model Training:

After preprocessing and selecting features, the dataset is divided into two parts:

- **Training dataset** – used to train the model
- **Testing dataset** – used to evaluate prediction accuracy

A **Random Forest Regression** model is then trained using the training data. Random Forest is an ensemble learning method that builds multiple decision trees and combines their outputs to produce more accurate predictions.

This method is chosen because:

- Handles nonlinear relationships effectively
- Works well with multiple input variables
- Reduces overfitting compared to a single decision tree
- Provides feature importance analysis

4.2.4 Prediction Process

Before showing the result, the system follows some steps in the background.

- First, the trained model is connected to the web application.
- When a student fills the form, the system checks if the inputs are correct.
- The values are then converted into numbers so the machine learning model can understand them.
- These values are sent to the Random Forest model.
- The model compares the inputs with patterns it learned from previous data.
- Finally, it calculates and shows the predicted exam score.

This whole process takes only a few seconds, so students get quick feedback about their performance.

4.2.5 Output Visualization:

The predicted exam score is displayed clearly on the web interface in percentage form.
The result appears in a highlighted box so that students can easily notice it.
The interface is simple and user-friendly, allowing users to understand their performance quickly. In future, the system may include graphs or suggestions for improvement.

Chapter 5

REQUIREMENT ANALYSIS

5.1 Introduction

Requirement analysis is an important phase in system development that defines the resources needed for the successful implementation of the project. It includes identifying both hardware and software requirements necessary for developing, testing, and deploying the system.

The AcadeScore system is designed to be lightweight, efficient, and easily deployable using standard computing resources. This chapter provides a detailed analysis of the hardware and software requirements needed for the system.

5.2 Hardware Requirements

The Hardware requirements refer to the physical components needed to run the system effectively.

5.2.1 Processor (CPU)

- Minimum Requirement: Intel Core i3 or equivalent
- Recommended Requirement: Intel Core i5 or higher

The processor plays a crucial role in executing the machine learning model and handling backend operations. Although the Random Forest algorithm is efficient, a better processor ensures faster computation and smooth execution.

5.2.2 Memory (RAM)

- Minimum Requirement: 4 GB
- Recommended Requirement: 8 GB or higher

RAM is required for processing datasets and running the web application. Machine learning models require sufficient memory for training and prediction tasks. Higher RAM improves system performance and reduces execution time.

5.2.3 Storage (Hard Disk)

- Minimum Requirement: 500 MB free space
- Recommended Requirement: 1 GB or more

Storage is required to store:

- Dataset files
- Machine learning model (.pkl file)
- Project code
- Output files

The storage requirement is relatively low as the system is lightweight.

5.2.4 Input Devices

- Keyboard
- Mouse

These devices are required for entering user data into the system.

5.2.5 Display

- Standard monitor (HD recommended)

A display is required to view the web interface and output results.

5.2.6 System Compatibility

- 32-bit or 64-bit system
- 64-bit preferred for better performance

5.3 Software Requirements

Software requirements include the tools, frameworks, and programming languages used to develop and run the system.

5.3.1 Operating System

The system is platform-independent and can run on:

- Windows (10/11)
- Linux (Ubuntu, Fedora)
- macOS

The use of Python ensures compatibility across multiple platforms.

5.3.2 Programming Language

- **Python**

Python is used due to its simplicity, readability, and extensive support for machine learning libraries.

5.3.3 Machine Learning Libraries

The following libraries are used:

- **Pandas:** Data handling and manipulation
- **NumPy:** Numerical computations
- **Scikit-learn:** Machine learning model implementation

These libraries provide powerful tools for data analysis and model training.

5.3.4 Web Framework

- **Flask**

Flask is used to develop the backend of the system. It handles user requests, processes input data, and communicates with the machine learning model.

5.3.5 Frontend Technologies

- HTML
- CSS
- JavaScript

These technologies are used to design the user interface and ensure smooth interaction between the user and the system.

5.3.6 Development Tools

- Visual Studio Code / PyCharm
- Jupyter Notebook (for model training)

These tools help in writing, debugging, and testing the code.

5.3.7 Browser Requirements

- Google Chrome
- Mozilla Firefox
- Microsoft Edge

A web browser is required to access the web application.

5.4 Functional Requirements

Functional requirements describe what the system should do.

The system must:

- Accept user input data

- Process the input data
- Predict student performance
- Display the result
- Provide a user-friendly interface

5.5 Non-Functional Requirements

Non-functional requirements describe how the system performs.

Performance

- The system should provide fast predictions

Usability

- The interface should be easy to use

Scalability

- The system should handle multiple users

Reliability

- The system should produce accurate results

These technologies are used to design the user interface and ensure smooth interaction between the user and the system.

Chapter 6

IMPLEMENTATION

6.1 Introduction

The implementation phase of the AcadeScore project focuses on converting the designed machine learning framework into a working system. This stage involves preparing the dataset, training the prediction model, and integrating the trained model into a web application.

The system is developed using Python for model building and Flask for the web interface. The goal of implementation is to ensure that the prediction process works accurately and that users can easily interact with the system through a simple browser interface.

The final system allows students to enter their lifestyle details and instantly receive a predicted exam performance percentage. Preprocessing techniques such as handling missing values, outlier detection, and feature scaling are applied to improve data quality and enhance model performance. The cleaned dataset is then divided into training and testing sets to enable proper model learning and unbiased evaluation. Furthermore, visualization and feature importance analysis are incorporated to interpret how each lifestyle factor contributes to academic performance. This implementation ensures that the system not only provides accurate predictions but also generates meaningful insights that can support students and educators in making informed decisions.

6.2 Machine Learning Algorithm Used

The AcadeScore system uses the Random Forest Regression algorithm to predict student performance.

Random Forest is an ensemble learning method that builds multiple decision trees using different parts of the dataset and combines their results to produce a more accurate prediction. Instead of depending on one tree, it averages the outputs of many trees, which improves stability and accuracy.

This algorithm is selected because it:

- Handles multiple input factors effectively
- Works well with nonlinear relationships in data

- Reduces overfitting compared to a single decision tree
- Provides feature importance to show which factors affect performance most
- Gives reliable predictions even with a moderate dataset

During training, the model learns the relationship between lifestyle factors and student scores. Once trained, the model is saved and integrated into the web application to generate predictions whenever a student enters their details.

6.3 System Architecture

6.3.1 User Enters Data

- The student opens the web page.
- A form is shown where the user enters details like:
- study hours, attendance, sleep, mental health, social media use, and job status.
- After filling the form, the user clicks the submit button.
- The frontend sends this data to the backend server.

6.3.2 Data Processing

- The Flask server receives the input data.
- It checks whether the values are correct and complete.
- The backend converts all inputs into numerical format.
- The processed data is then sent to the trained machine learning model.

6.3.3 Prediction (Model Layer – Random Forest)

- The Random Forest model receives the processed inputs.
- It compares them with patterns learned during training.
- The model calculates the expected exam score.
- The predicted performance percentage is generated.

6.3.4 Result Sent Back to Server

- The model sends the predicted score back to the Flask backend.
- The backend prepares the result for display.

6.3.5 Result Displayed to User

- The backend sends the result to the webpage.
- The frontend shows the predicted score clearly in percentage form.
- The student can instantly see their expected performance.

System Architecture DIG:

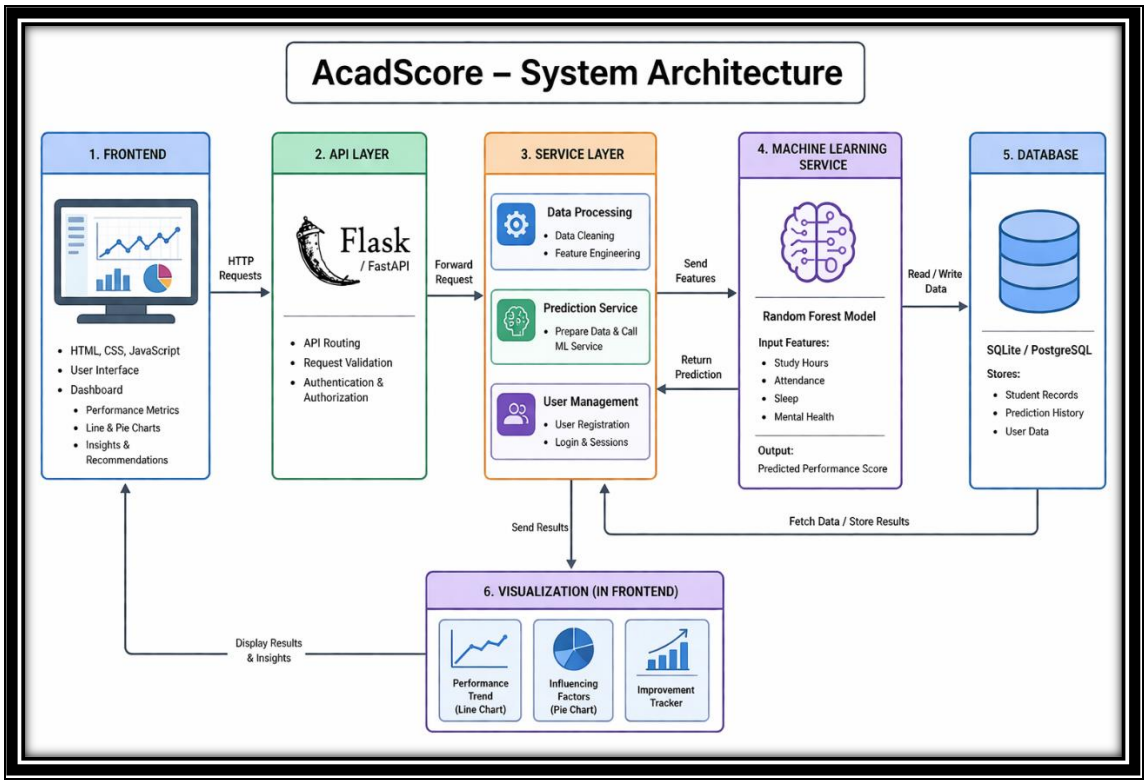


Fig 6.3: AcadeScore System Architecture

6.4 Implementation Procedure

The implementation of the system is carried out in the following steps:

i. Dataset Preparation

The dataset is first collected and organized into tabular form.

Preprocessing steps include:

- Removing missing or incorrect values
- Converting categorical values into numerical form
- Selecting relevant features
- Splitting the dataset into training and testing sets
- This ensures that the dataset is suitable for model training.

ii. Model Training

The Random Forest Regression model is trained using the prepared dataset.

The model learns the relationship between lifestyle factors and student performance.

Model training steps include:

- The model is evaluated using MAE, RMSE, and R^2 score
- Accuracy is checked using the testing dataset
- The trained model is saved as a file for later use

iii. Web Integration

Once the model is ready, it is integrated into a Flask web application.

Steps include:

- Creating an HTML form for user input
- Writing Flask routes to handle requests
- Loading the trained model in the backend
- Passing input data to the model
- Displaying prediction results on the webpage
- This step converts the machine learning model into a usable web-based system.

iv. Testing

Finally, the system is tested to ensure correct functionality.

Testing includes:

- Checking if the web form accepts valid inputs
- Verifying that the model generates predictions correctly
- Comparing predictions with expected outputs
- Ensuring the interface displays results clearly
- Successful testing confirms that the AcadeScore system works properly and is ready for use.

Chapter 7

RESULTS AND DISCUSSION

7.1 Introduction

The results and discussion chapter presents the outcomes obtained from the implementation of the AcadeScore system. It evaluates the performance of the machine learning model and analyzes how effectively the system predicts student performance.

This chapter also discusses the behavior of the system under different input conditions and verifies the correctness of the prediction process.

7.2 System Execution

The AcadeScore system was successfully implemented and tested using a web-based interface. The application allows users to input student-related data such as study hours, attendance percentage, sleep duration, and mental health rating.

The system processes the input data and uses the trained Random Forest Regression model to generate predictions.

Execution Steps:

i. User opens the web application
ii. Inputs required data in the form
iii. Submits the form
iv. Data is sent to Flask backend
v. Backend processes input
vi. Model predicts performance
vii. Output is displayed

The system executes efficiently without delays, demonstrating smooth integration between frontend, backend, and machine learning components.

7.3 Output Results

The system generates output in the form of predicted student performance.

Example Output:

- Input:
    - Study Hours: 5
    - Attendance: 85%

  - Sleep: 7 hours
  - Mental Health: Good
- Output:

  Predicted Score: 78

The results are displayed on the dashboard in a clear and understandable format.

7.4 Performance Analysis

The performance of the model was evaluated using standard metrics:

- **Mean Absolute Error (MAE)**

  Measures the average difference between predicted and actual values.
- **Mean Squared Error (MSE)**

  Measures squared differences to penalize larger errors.
- **R^2 Score**

  Indicates how well the model fits the data.

  The model showed satisfactory performance with good prediction accuracy.

7.5 Discussion of Results

The results indicate that the system can effectively predict student performance based on input data.

Key Observations:

- Study hours and attendance have a strong impact on performance
- Lifestyle factors such as sleep and mental health also influence results
- The model handles multiple features efficiently

The system provides realistic predictions, which can help students understand their academic standing.

7.6 Conclusion

The results confirm that the AcadeScore system successfully predicts student performance using machine learning techniques. The system is efficient, accurate, and user-friendly.

Chapter 8

CONCLUSION

8.1 Conclusion

The AcadeScore system demonstrates the successful application of machine learning in the field of education. The project was designed to predict student performance based on academic and lifestyle factors, and it achieves this objective effectively.

The system uses the Random Forest Regression algorithm, which provides accurate and reliable predictions. The integration of the machine learning model with a web-based interface makes the system user-friendly and accessible.

Throughout the development process, various phases such as data collection, preprocessing, model training, evaluation, and deployment were completed successfully. The system is capable of analysing multiple input features and generating meaningful predictions.

The project highlights the importance of data-driven approaches in education. By providing early predictions, the system helps students identify their weaknesses and improve their performance.

Overall, the AcadeScore system is an efficient and practical solution for student performance prediction. However, it is important to acknowledge that the accuracy of the model depends on the quality and size of the dataset. A more diverse and larger dataset could further improve prediction accuracy. Additionally, incorporating more features such as socio-economic factors or academic history could enhance the effectiveness of the system.

Overall, the AcadeScore system highlights the importance of data-driven approaches in modern education. It provides a foundation for developing intelligent educational tools that can assist students in achieving better academic performance. The project not only demonstrates technical implementation but also emphasizes the real-world impact of machine learning in improving educational outcomes.

8.2 Future Enhancements

Although the current system performs well, several improvements can be made in future versions to increase its functionality and impact.

Future enhancements aim to address the limitations of the current system, such as dependency on a limited dataset, absence of real-time tracking, and lack of personalized feedback. By overcoming these limitations, the system can provide deeper insights, better accuracy, and more meaningful recommendations to students.

In addition, the rapid development of artificial intelligence and deep learning technologies offers new possibilities for improving prediction models. These advancements can enable the system to learn more complex patterns and provide highly accurate and personalized predictions.

- **Mobile Application Development**
  The system can be extended to a mobile application to provide better accessibility for users.
- **Real-Time Data Tracking**
  The system can be enhanced to track student data over time and provide continuous performance updates.
- **Advanced Machine Learning Models**
  More advanced algorithms such as deep learning models can be used to improve prediction accuracy.
- **Integration with Educational Systems**
  The system can be integrated with school or college databases to automatically fetch student data.
- **Personalized Recommendations**
  The system can provide suggestions to students on how to improve their performance.
- **Data Visualization Dashboard**
  Graphs and charts can be added to visualize student progress and trends.

REFERENCES

[1] L. R. Pelima, Y. Sukmana, and Y. Rosmansyah,
"Predicting University Student Graduation Using Academic Performance and Machine Learning: A Systematic Literature Review,"
IEEE Access, vol. 12, pp. 23451–23465, 2024,
doi: 10.1109/ACCESS.2024.3361479.

[2] E. Alhazmi and A. Sheneamer,
"Early Predicting of Students Performance in Higher Education,"
IEEE Access, vol. 11, pp. 27579–27590, 2023,
doi: 10.1109/ACCESS.2023.3250702.

[3] M. Khan et al.,
"Utilizing Machine Learning Models to Predict Student Performance From LMS Activity Logs,"
IEEE Access, vol. 11, pp. 86953–86965, 2023,
doi: 10.1109/ACCESS.2023.3305276.

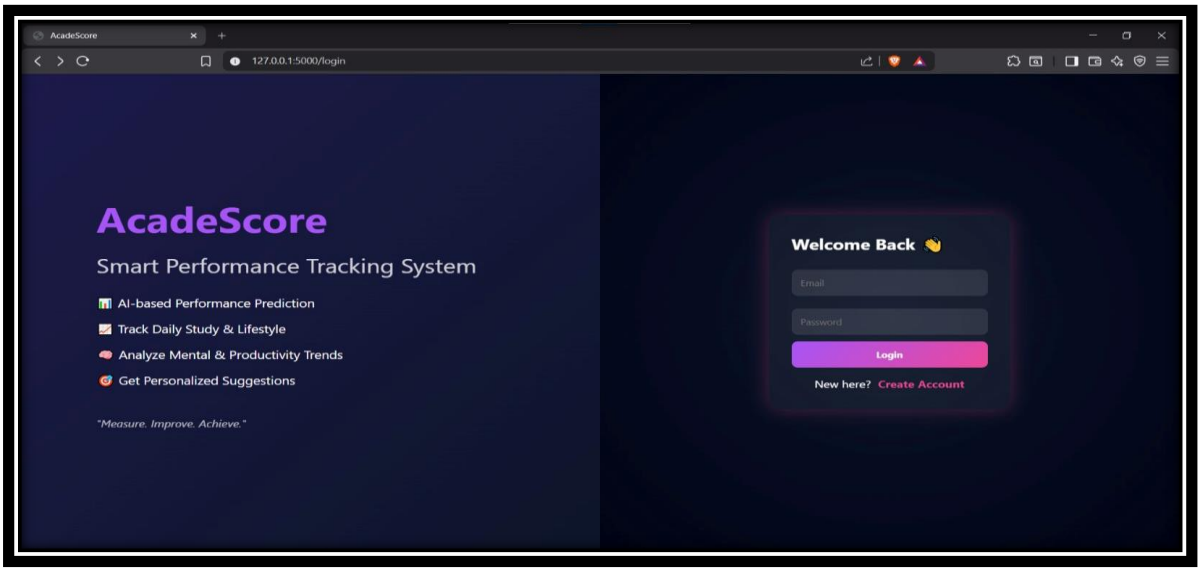
[4] S. D. Abdul Bujang et al.,
"Multiclass Prediction Model for Student Grade Prediction Using Machine Learning,"
IEEE Access, vol. 9, pp. 95608–95620, 2021,
doi: 10.1109/ACCESS.2021.3093563.

[5] A. Alshanqiti and A. Namoun,
"Predicting Student Performance and Its Influential Factors Using Hybrid Regression and Multi-Label Classification,"
IEEE Access, vol. 8, pp. 203827–203840, 2020,
doi: 10.1109/ACCESS.2020.3036572.

Github repository: https://github.com/alimusharraf/AcadeScore

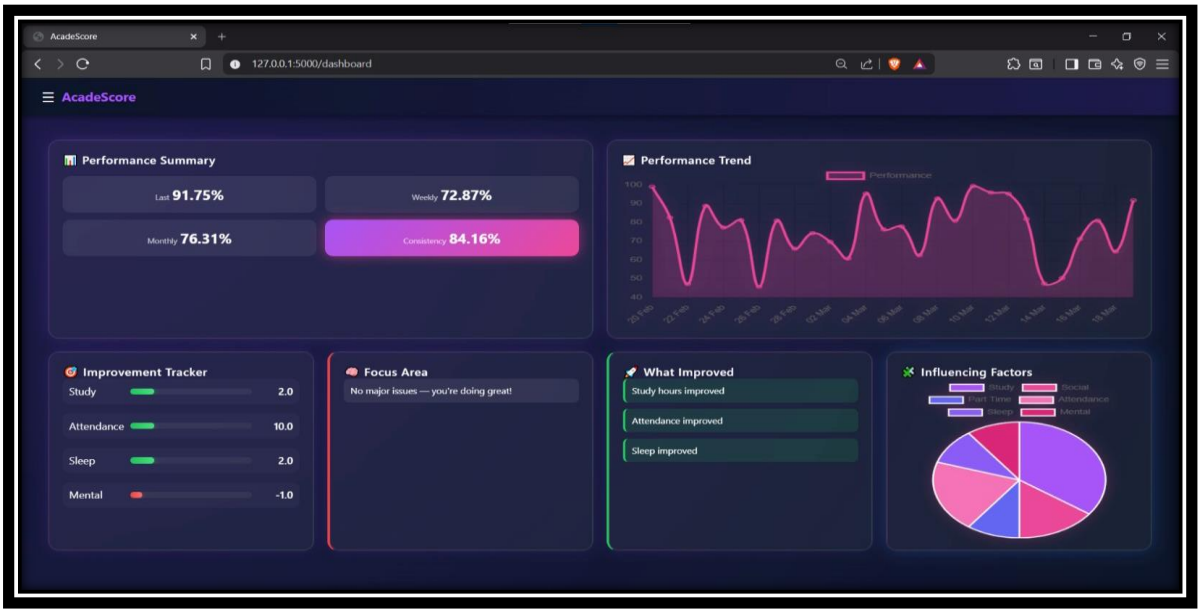
APPENDIX A

SCREENSHOTS



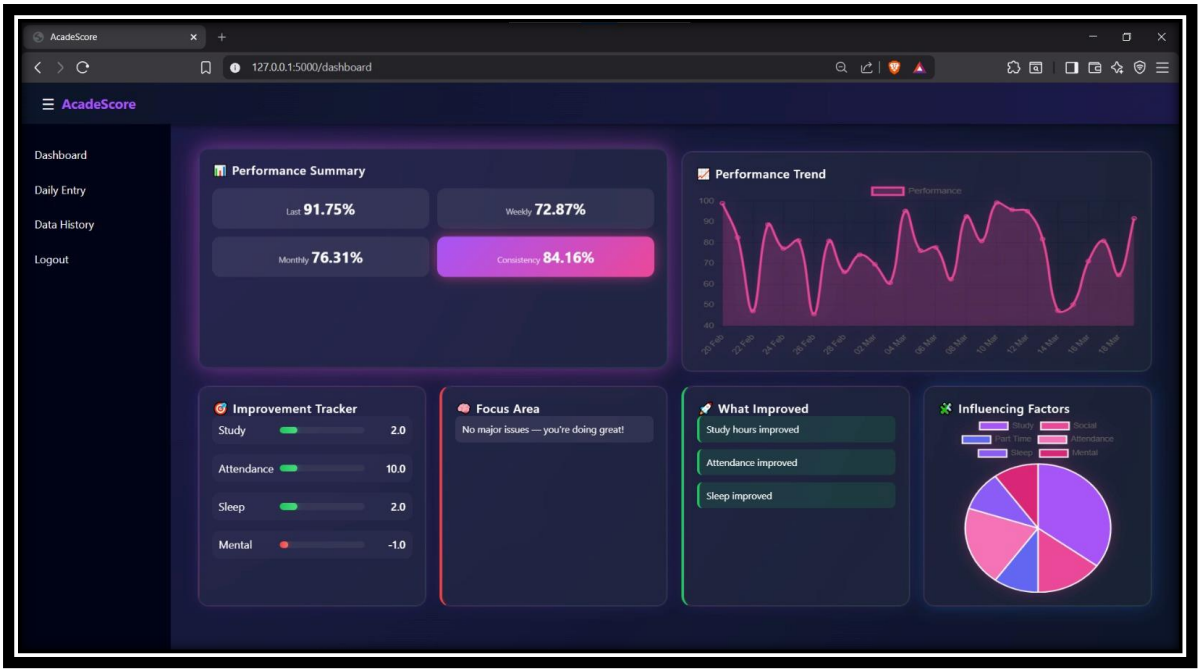
Screenshot 1: Login Page

The login page enables secure user authentication through email and password input. It also highlights key features like AI-based performance prediction, daily activity tracking, and personalized suggestions, giving users a quick overview of the system.



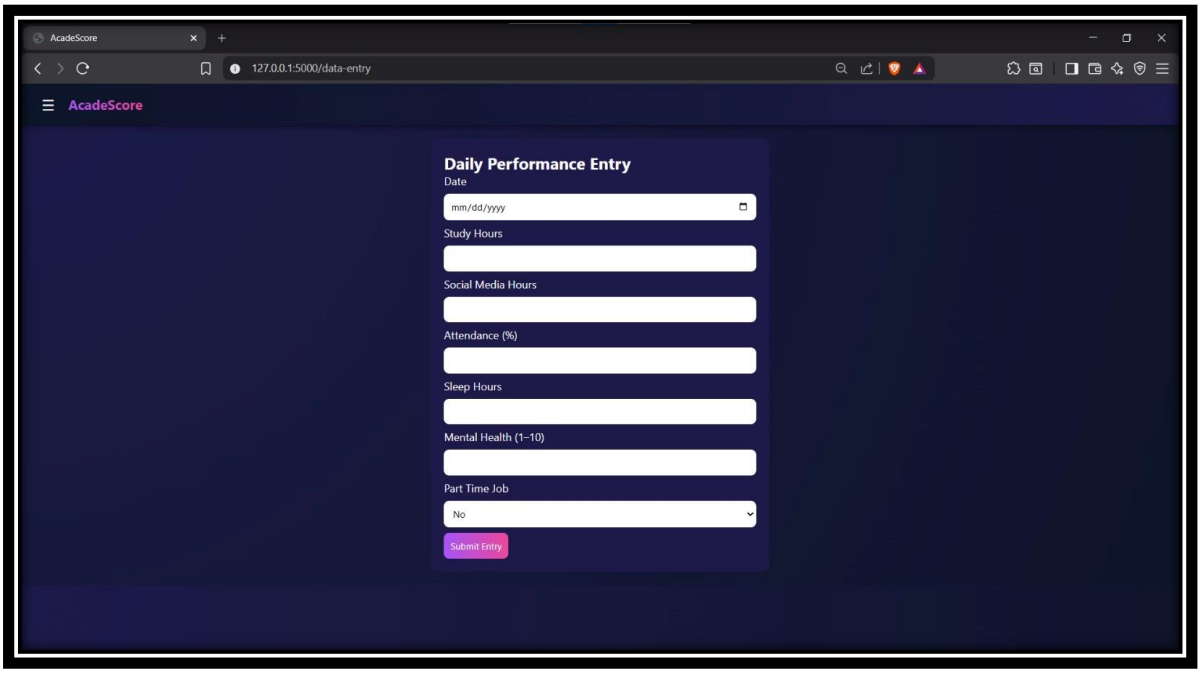
Screenshot 2: Dashboard (Overview & Analytics)

The dashboard provides a detailed overview of user performance using summary cards, line graphs, and charts. It helps users analyse trends, monitor consistency, and understand how different factors impact their overall performance.



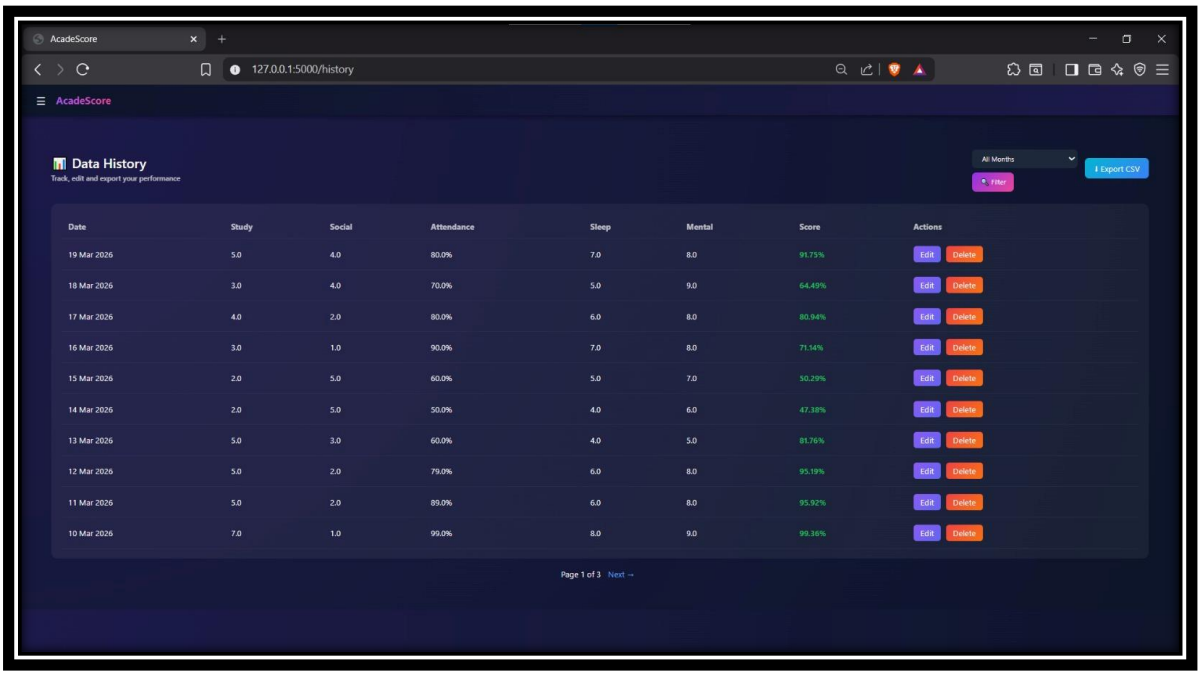
Screenshot 3: Dashboard with Navigation Panel

This screen includes a sidebar navigation panel that allows users to easily switch between different modules such as dashboard, daily entry, and data history, improving accessibility and overall user experience within the system.



Screenshot 4: Daily Performance Entry Form

The daily entry form allows users to input important data such as study hours, attendance percentage, sleep, social media usage, and mental health, which is used by the system for analysis and performance prediction.



Screenshot 5: Data History Screen

The data history screen displays all recorded entries in a structured table format with options to edit, delete, filter, and export data, helping users efficiently manage and review their past performance records.

APPENDIX B

SOURCE CODE

```
from flask import Flask, render_template, request, redirect, jsonify, make_response
from flask_sqlalchemy import SQLAlchemy
from flask_login import LoginManager, login_user, login_required, logout_user, UserMixin, current_user
from werkzeug.security import generate_password_hash, check_password_hash
import joblib
import pandas as pd
import numpy as np
from datetime import datetime, date, timedelta
from sklearn.linear_model import LinearRegression
from sqlalchemy import extract
import csv
from io import StringIO
from statistics import mean, stdev

app = Flask(__name__)
app.config["SECRET_KEY"] = "secret"
app.config["SQLALCHEMY_DATABASE_URI"] = "sqlite:///acadescore.db"

db = SQLAlchemy(app)
login_manager = LoginManager(app)
login_manager.login_view = "login"

model = joblib.load("Random_Forest.pkl")

# ---------------- DATABASE MODELS ----------------

class User(UserMixin, db.Model):
    id = db.Column(db.Integer, primary_key=True)
    email = db.Column(db.String(100), unique=True)
    password = db.Column(db.String(200))

class Prediction(db.Model):
    id = db.Column(db.Integer, primary_key=True)
    user_id = db.Column(db.Integer, nullable=False)

    study_hours = db.Column(db.Float)
    social_media = db.Column(db.Float)
    part_time_job = db.Column(db.Integer)
```

```
    attendance = db.Column(db.Float)
    sleep = db.Column(db.Float)
    mental_health = db.Column(db.Float)

    score = db.Column(db.Float)

    entry_date = db.Column(db.Date, nullable=False)

@login_manager.user_loader
def load_user(user_id):
    return User.query.get(int(user_id))

@app.route("/")
def home():
    return redirect("/login")

@app.route("/login", methods=["GET", "POST"])
def login():
    error = None
    if request.method == "POST":
        user = User.query.filter_by(email=request.form["email"]).first()

        if not user:
            error = "Account does not exist. Please register."
        elif not check_password_hash(user.password, request.form["password"]):
            error = "Incorrect password."
        else:
            login_user(user)
            return redirect("/dashboard")

    return render_template("login.html", error=error)

@app.route("/register", methods=["GET", "POST"])
def register():
    error = None
    if request.method == "POST":
        existing = User.query.filter_by(email=request.form["email"]).first()
        if existing:
            error = "Email already registered."
        else:
            hashed = generate_password_hash(request.form["password"])
            user = User(email=request.form["email"], password=hashed)
            db.session.add(user)
            db.session.commit()
```

```
        return redirect("/login")

    return render_template("register.html", error=error)

@app.route("/logout")
@login_required
def logout():
    logout_user()
    return redirect("/login")

# ---------------- DATA ENTRY ----------------
@app.route("/data-entry", methods=["GET", "POST"])
@login_required
def data_entry():

    if request.method == "POST":

        # Get selected date
        entry_date = datetime.strptime(
            request.form["entry_date"],
            "%Y-%m-%d"
        ).date()

        # Prevent future date
        if entry_date > date.today():
            return render_template(
                "data_entry.html",
                error="Future dates are not allowed.",
                today=date.today()
            )

        # Check duplicate entry for same date
        existing_entry = Prediction.query.filter_by(
            user_id=current_user.id,
            entry_date=entry_date
        ).first()

        if existing_entry:
            return render_template(
                "data_entry.html",
                error="Entry already exists for selected date.",
                today=date.today()
            )
```

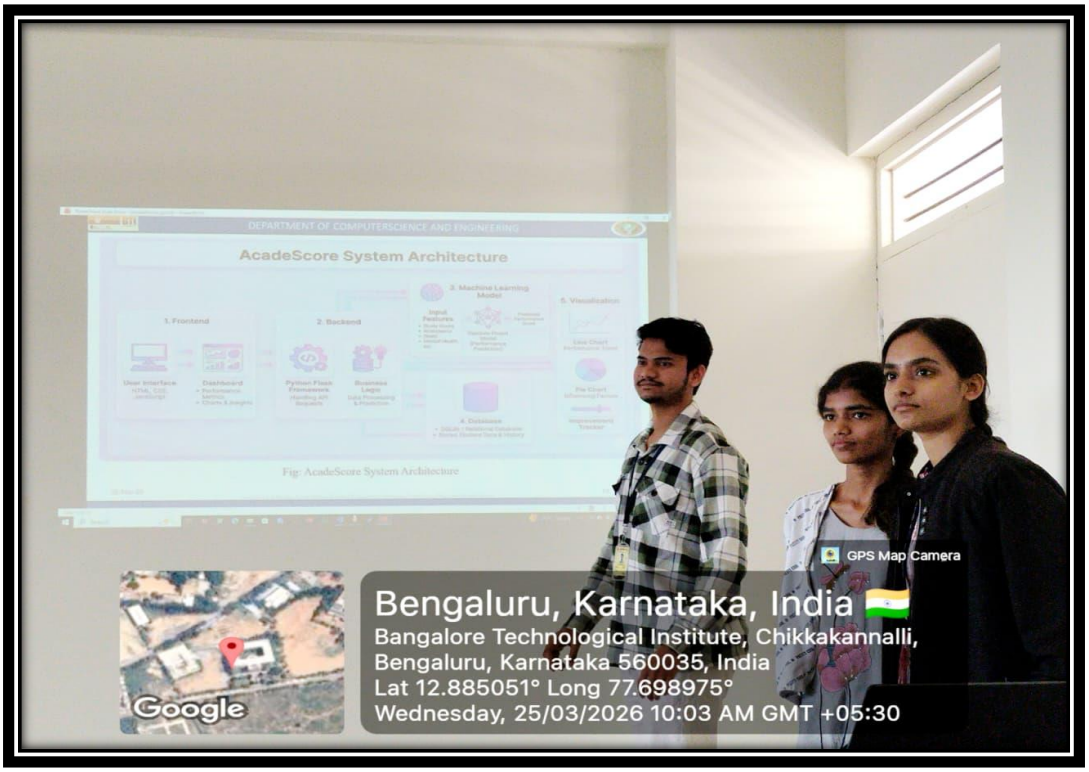
```
        # Prepare ML input (ONLY 6 FEATURES)
        input_df = pd.DataFrame([{
            "study_hours_per_day": float(request.form["study_hours"]),
            "social_media_hours": float(request.form["social_media"]),
            "part_time_job": int(request.form["part_time_job"]),
            "attendance_percentage": float(request.form["attendance"]),
            "sleep_hours": float(request.form["sleep"]),
            "mental_health_rating": float(request.form["mental_health"])
        }])

        predicted_score = model.predict(input_df)[0]

        # Save to database
        new_entry = Prediction(
            user_id=current_user.id,
            study_hours=float(request.form["study_hours"]),
            social_media=float(request.form["social_media"]),
            part_time_job=int(request.form["part_time_job"]),
            attendance=float(request.form["attendance"]),
            sleep=float(request.form["sleep"]),
            mental_health=float(request.form["mental_health"]),
            score=predicted_score,
            entry_date=entry_date
        )

        db.session.add(new_entry)
        db.session.commit()

        return redirect("/dashboard")

    return render_template(
        "data_entry.html",
        today=date.today()
    )
# ---------------- DASHBOARD ----------------

@app.route("/dashboard")
@login_required
def dashboard():

    # Get all entries sorted by date
    entries = Prediction.query.filter_by(
        user_id=current_user.id
    ).order_by(Prediction.entry_date.asc()).all()
```

```
# Latest entry
latest = entries[-1] if entries else None

# Prepare graph data
scores = [round(e.score, 2) for e in entries]
dates = [e.entry_date.strftime("%d %b") for e in entries]

# CONSISTENCY SCORE
if len(scores) > 1:
    std = np.std(scores)
    consistency = round(max(0, 100 - std), 2)
else:
    consistency = 100 if scores else 0

# ==============================
# IMPROVEMENT TRACKER
# ==============================

study_improvement = 0
attendance_improvement = 0
sleep_improvement = 0
mental_improvement = 0

if len(entries) >= 2:

    prev = entries[-2]
    curr = entries[-1]

    study_improvement = round(
        curr.study_hours - prev.study_hours, 2
    )

    attendance_improvement = round(
        curr.attendance - prev.attendance, 2
    )

    sleep_improvement = round(
        curr.sleep - prev.sleep, 2
    )

    mental_improvement = round(
        curr.mental_health - prev.mental_health, 2
    )
```

```
# WHAT IMPROVED

improvements = []

if study_improvement > 0:
    improvements.append("Study hours improved")

if attendance_improvement > 0:
    improvements.append("Attendance improved")

if sleep_improvement > 0:
    improvements.append("Sleep improved")

if mental_improvement > 0:
    improvements.append("Mental health improved")

# SUGGESTIONS
suggestions = []
if latest:

    if latest.study_hours < 4:
        suggestions.append("Increase study hours")

    if latest.attendance < 75:
        suggestions.append("Improve attendance")

    if latest.sleep < 6:
        suggestions.append("Sleep more for better performance")

    if latest.mental_health < 6:
        suggestions.append("Improve mental wellness")

    if latest.social_media > 5:
        suggestions.append("Reduce social media usage")

    # Recalculate prediction
    input_df = pd.DataFrame([{
        "study_hours_per_day": entry.study_hours,
        "social_media_hours": entry.social_media,
        "part_time_job": entry.part_time_job,
        "attendance_percentage": entry.attendance,
        "sleep_hours": entry.sleep,
        "mental_health_rating": entry.mental_health
```

```
        }])

        entry.score = model.predict(input_df)[0]

        db.session.commit()

        return redirect("/history")

    return render_template("edit.html", entry=entry)

@app.route("/export")
@login_required
def export_csv():

    entries = Prediction.query.filter_by(
        user_id=current_user.id
    ).all()

    output = StringIO()
    writer = csv.writer(output)

    writer.writerow(["Date","Study","Social","Attendance","Sleep","Mental","Score"])

    for e in entries:
        writer.writerow([
            e.entry_date.strftime("%d-%m-%Y"),
            e.study_hours,
            e.social_media,
            e.attendance,
            e.sleep,
            e.mental_health,
            e.score
        ])

    response = make_response(output.getvalue())
    response.headers["Content-Disposition"] = "attachment; filename=data.csv"
    response.headers["Content-type"] = "text/csv"

    return response

if __name__ == "__main__":
    with app.app_context():
        db.create_all()
    app.run(debug=True)
```

PHOTO GALLERY



AcadeScore Project Presentation